FLUTTER LINUX WEB PORTAL

App-gated website access using Flutter + Python FastAPI

Architecture • Request flow • Setup • Security hardening

Implementation guide

Prepared 22 October 2026

Purpose

This guide describes a pattern for opening a website inside a Linux Flutter desktop app while making unauthenticated direct visits return an empty response. A Python FastAPI backend issues a short-lived, single-use launch ticket and exchanges it for a browser session. The guide explains the flow and gives a local development setup, then highlights what must change before production deployment.

> **Important:** This is an access-control pattern, not proof that a request came from an untampered Flutter binary. A secret embedded in a desktop application can be extracted. Do not use a hard-coded shared key as the only production security control.

1. System overview

The website and its protected APIs must enforce authorization on the server. Hiding a page in Flutter alone does not prevent someone from opening the URL in another browser.

Component	Responsibility
Flutter Linux app	Requests a launch ticket from the backend, opens the portal in an embedded WebView, and restricts navigation to the configured website host.
Python FastAPI backend	Validates the launch credential, issues a short-lived one-time ticket, consumes the ticket, and establishes a session.
Website / protected routes	Return the actual page only for a valid session; return an empty response or deny access otherwise.
HTTPS reverse proxy	Terminates TLS, forwards traffic to FastAPI, and applies deployment-level limits and headers.

High-level request flow

1. Flutter app Requests a launch ticket	**2. FastAPI** Checks app credential	**3. One-time ticket** Short expiry
4. WebView Opens app-entry with ticket	**5. Backend** Consumes ticket once	**6. Session cookie** Redirect to portal

Access outcomes

- **Valid launch:** a valid ticket is exchanged for a session; the WebView can then load the protected portal.
- **Direct browser visit without a session:** the server returns an empty 404 response for the protected page, if that is the intended user experience.
- **Protected API without a session:** the server returns 401 Unauthorized (or another deliberate denial response).
- **Expired or reused ticket:** reject it; tickets should be random, short-lived, and single-use.

A blank page is only a user-interface behavior. The real protection is server-side authorization on every protected page and API.

2. Detailed workflow

1 — Start the app

The Flutter desktop app starts and reads its configured backend URL. It should accept navigation only to the intended portal host.

2 — Request a launch ticket

Flutter sends a POST request to /api/app-ticket with the configured launch credential in the X-App-Key header. The backend validates the credential and creates a cryptographically random ticket with a short lifetime, for example 60 seconds.

3 — Open the portal entry route

Flutter opens /app-entry?ticket=... in its embedded WebView. The ticket should be treated as a temporary bearer credential and must not be logged.

4 — Consume the ticket

The backend verifies that the ticket exists, has not expired, and has not already been used. It invalidates the ticket immediately after successful consumption.

5 — Create a session

The backend creates a session and sets a cookie. Configure the cookie with HttpOnly and SameSite protections; use Secure=true over HTTPS. The backend redirects the WebView to the portal root.

6 — Serve protected content

The portal route checks the session. A valid session receives the website; a missing or invalid session receives an empty 404 response (if intentionally configured). Protected API routes check the same session and return 401 when unauthorized.

7 — Expire or revoke access

The session expires after its configured lifetime, such as eight hours, or is revoked server-side. The app should be able to recover gracefully by requesting a fresh ticket and signing in again.

3. Project files and their roles

Path	Purpose / review points
backend/app.py	FastAPI routes for ticket creation, ticket exchange, session validation, portal response, and protected API examples.
backend/requirements.txt	Python dependencies such as FastAPI and Uvicorn. Pin reviewed versions for repeatable deployments.
backend/.env.example	Example environment settings. Keep real secrets out of source control.
flutter_app/lib/main.dart	Flutter UI, ticket request, embedded WebView setup, and host allow-list. Do not assume a bundled secret is confidential.
flutter_app/pubspec.yaml	Flutter dependencies, including the WebView and HTTP client packages.
nginx/portal.conf	Example reverse-proxy configuration for HTTPS and forwarding traffic to the backend. Adapt to the actual domain and deployment.
README.md	Project setup, run instructions, test flow, and security notes.

Suggested responsibilities

- Flutter: app navigation, WebView presentation, ticket request, and friendly error handling.
- Backend: all authorization decisions, ticket expiry and single-use enforcement, session lifecycle, and protected route checks.
- Reverse proxy / deployment: TLS, request-size limits, rate limiting, trusted proxy configuration, and secure operational logging.

The backend must not rely on a Flutter-only check. Users can bypass the app and send HTTP requests directly to the server.

4. Local development setup

These commands illustrate a local development run. They assume the project contains the backend and Flutter folders described above.

A. Configure and run the FastAPI backend

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Generate two different random values:
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

Set the first generated value as SESSION_SECRET and the second as APP_LAUNCH_KEY. For local HTTP testing only:

```
export SESSION_SECRET='PASTE_FIRST_RANDOM_VALUE'
export APP_LAUNCH_KEY='PASTE_SECOND_RANDOM_VALUE'
export COOKIE_SECURE=false
export ALLOWED_HOSTS='localhost,127.0.0.1'

uvicorn app:app --host 127.0.0.1 --port 8000
```

B. Configure and run Flutter

```
cd flutter_app
flutter pub get
flutter run -d linux
```

Before running, check the app configuration

- Set baseUrl to the backend URL used in this environment.
- Set allowedHost to the exact portal hostname and enforce HTTPS in production.
- Confirm the Flutter ticket-request header name and backend validation logic match exactly.
- Ensure the WebView cookie jar is retained between the ticket exchange and loading the portal.
- Never commit real secrets to Git; use environment-specific configuration and rotate exposed keys.

Local development can use HTTP and COOKIE_SECURE=false. Production should use HTTPS and COOKIE_SECURE=true; do not copy the local settings into production.

5. Verification checklist

Test	Expected result
Open the protected portal URL in a clean browser session	Empty 404 / blank response if that behavior is intentionally configured.
Call a protected API without a session cookie	401 Unauthorized.
Launch from the Flutter app with a valid credential	Ticket endpoint succeeds; WebView obtains a session and loads the portal.
Use an invalid launch credential	Ticket endpoint rejects the request; no session is created.
Try an expired ticket	Ticket exchange is rejected.
Try to reuse a consumed ticket	Second exchange is rejected.
Navigate the WebView to an unrelated host	Flutter navigation policy blocks it.
Wait for session expiry / revoke the session	Protected content is no longer available until a new authorized session is established.
Inspect server logs	No launch tickets, cookies, API keys, or session identifiers are written to logs.

Useful manual checks

```
# Check that the backend is listening locally:
curl -i http://127.0.0.1:8000/

# Check a protected API without a session:
curl -i http://127.0.0.1:8000/api/protected-example
```

Exact responses depend on the routes implemented in your app. Adapt the endpoint names in these examples to match the actual code.

6. Security limitations and production hardening

The starter design is useful for controlling ordinary access, but it should not be described as making a website impossible to open outside the Flutter app.

Key limitations

- **Embedded app key:** a static key packaged in a desktop binary can be extracted. A user who obtains it may imitate the ticket request.
- **Ticket in URL:** query-string tickets can leak through access logs, browser diagnostics, crash reports, or referrer handling. Keep their lifetime short and redact them; a POST-based exchange is preferable for production.
- **Client checks are not authorization:** a host allow-list in Flutter improves user experience but does not secure the backend. Every protected route must validate the session server-side.
- **Session storage:** in-memory ticket/session stores may be lost on restart and do not work reliably across multiple backend workers. Use Redis or a database for shared, expiring state.

Production checklist

- Serve the portal and API only over HTTPS; set Secure, HttpOnly, and appropriate SameSite cookie attributes.
- Use high-entropy, single-use, short-lived tickets and store only a hash of each ticket where practical.
- Use server-side sessions with expiry, logout/revocation, rotation, and a shared store such as Redis when scaling.
- Add rate limiting, request validation, security headers, audit events, and monitoring for repeated failures.
- Protect state-changing cookie-authenticated endpoints against CSRF; validate Origin/Host where appropriate.
- Avoid logging secrets, tickets, session cookies, or full authorization headers.
- Use per-device credentials or a proper user/device authentication flow rather than a single shared secret in the binary.
- Consider signed challenges or device identity controls if device enrollment is required. Hardware-backed storage can raise the bar but does not make a client fully unmodifiable.
- Test authorization for every HTML route, download, WebSocket, and API endpoint—not only the home page.

Threat model note

If the requirement is specifically “only this exact, untampered app process may access the site,” a static key is insufficient. Desktop clients are controlled by the person running them. Stronger controls require an explicit identity and trust model, such as authenticated user accounts, managed-device enrollment, signed requests, or enterprise device attestation where available.

7. Troubleshooting and deployment notes

Symptom	Likely checks
WebView stays blank after launch	Check backend logs without printing secrets; verify ticket endpoint response, ticket expiry, route names, redirect URL, cookie domain/path, and WebView cookie handling.
Ticket request returns 401/403	Confirm the configured launch credential matches the backend value and the header name is correct. Rotate any secret that may have been exposed.
Portal works in Flutter but not in a normal browser	This is expected when the browser has no valid session. Confirm the backend—not just Flutter—enforces this.
Session is not retained	Check cookie Secure/SameSite/Domain/Path attributes and ensure HTTP versus HTTPS configuration matches the environment.
Works with one worker but fails with multiple workers	Move ticket and session state into a shared Redis/database store rather than process-local memory.
External links open unexpectedly	Review the WebView navigation delegate/allow-list and decide how downloads, pop-ups, and external links should behave.

Deployment outline

- Configure DNS and a TLS certificate for the portal domain.
- Run FastAPI behind a production reverse proxy; bind the app server to a private interface or Unix socket.
- Set production secrets through the deployment secret manager/environment, not source files.
- Set COOKIE_SECURE=true, configure the actual allowed hosts, and review trusted proxy settings.
- Run negative tests from a separate browser and direct HTTP client before release.

This document describes the intended architecture and sample commands. Before deploying, compare every route, environment variable, and cookie setting against the actual project files.